

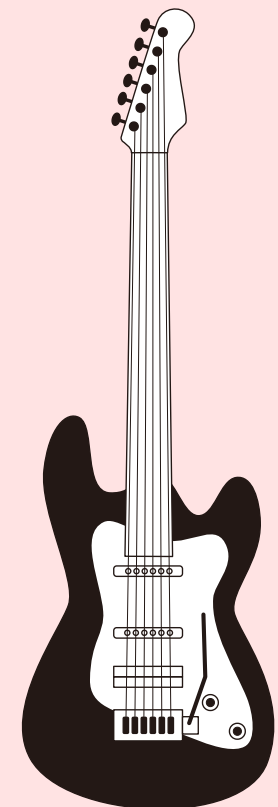
PRACTICAL DATABASE IMPLEMENTATION

FOR DATA ANALYSIS:

A Case Study of the Chinook Digital Music Store

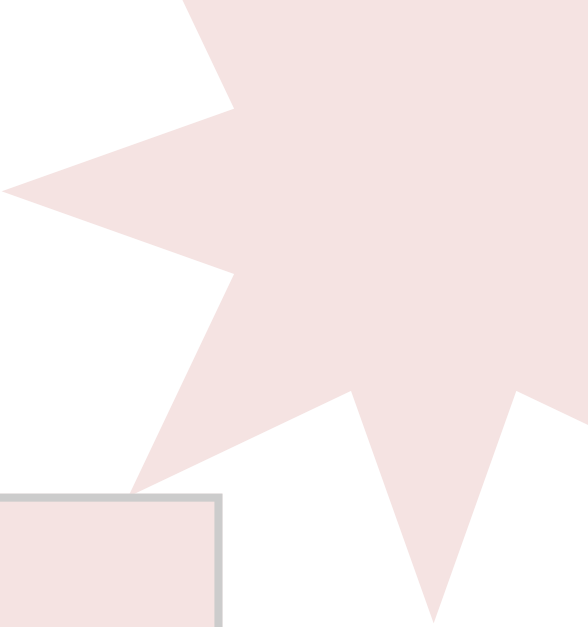
Presented by: D2NB

Lecturer: Mr. Dang Thai Doan





TEAM MEMBERS



NAME	IRN
Văn Vũ Quỳnh Như	2232300079
Vũ Đông Dương	2032300044
Thân Quế Ngọc	2232300060
Đỗ Hoàng Bảo	2232300071

TABLE OF CONTENTS

- ◆ — I. Project Overview & Business Scope
- ◆ — II. Database Design & Implementation
- ◆ — III. SQL Analytical Querying
- ◆ — IV. Python Connection & Data Visualization
- ◆ — V. Conclusion & Recommendations

I. PROJECT OVERVIEW & BUSINESS SCOPE

- **Database Selection:** Uses the Chinook sample database (originally by Luis Rocha).
- **Implementation Environment:** Re-established in PostgreSQL 17 using pgAdmin 4.
- **Structure:** Consists of 11 interconnected tables.
- **Data Content:** Includes both media inventory and business transactions.

11 Tables

7 Media Inventory
4 Business Transact

- artist, album, track...
- genre, media_type...
- customer, invoice...
- employee, playlist...

3,503

Media Tracks

347 Albums • 275 Artists
59 Customers • 8 Employees

Based on the
Chinook sample
dataset

- **Tech Stack:** Rebuilt in PostgreSQL 17 using pgAdmin 4.
- **Scope:** Proprietary analysis formulating distinct business questions (not replicating original scope).
- **Key Objective:** Apply custom analytical methodologies to extract insights on customer purchasing behavior and media performance.

EIU

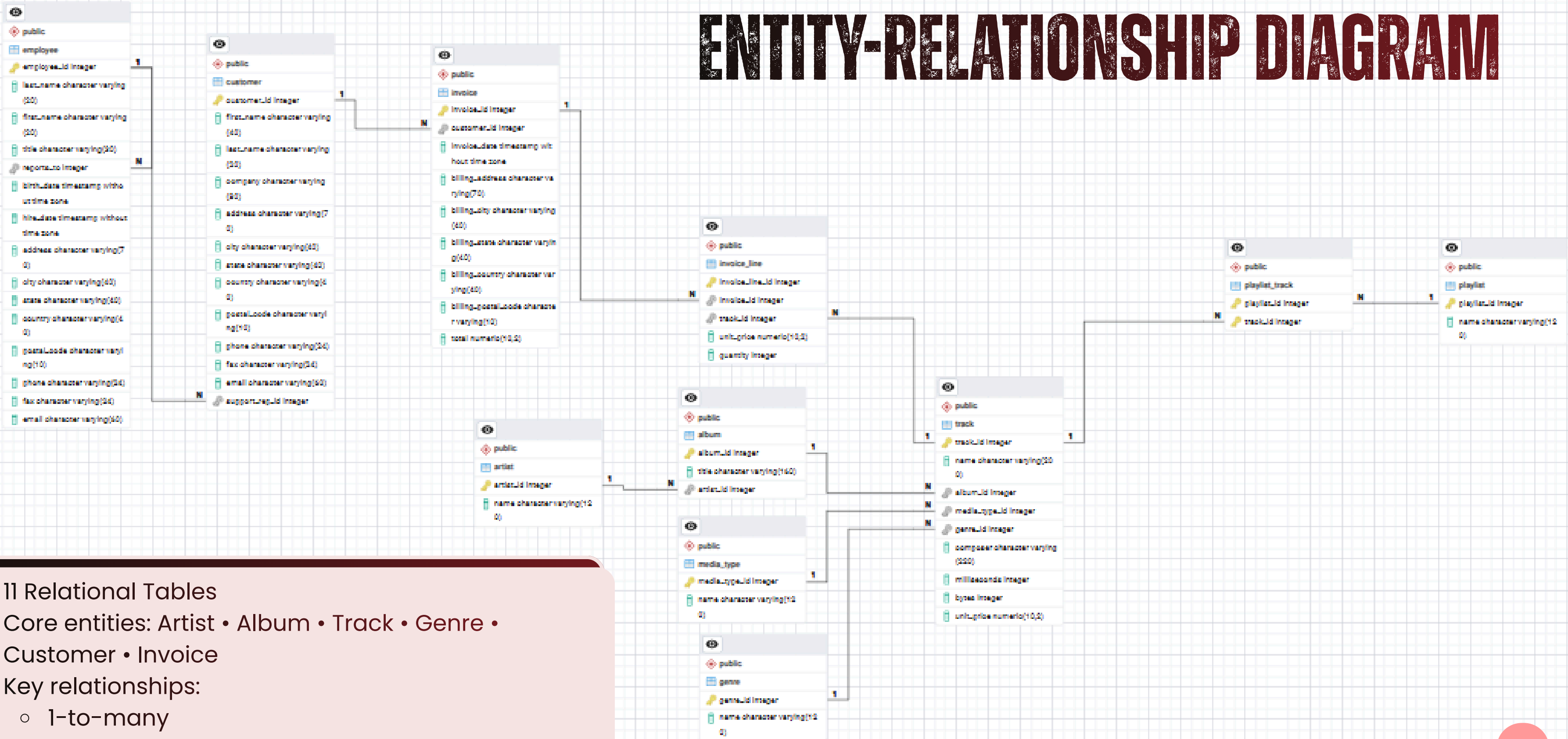
MEDIA INVENTORY TABLES

TABLE	WHAT IT STORES	PRIMARY KEY	FOREIGN KEYS
artist	Artist names	artist_id	none
album	Album titles and the associated artist	album_id	artist_id
media_type	Names of media file formats	media_type_id	none
genre	Names of music and video genres	genre_id	none
track	Media track details (name, length, size, unit price, album, media type, genre)	track_id	album_id, media_type_id, genre_id
playlist	Playlist names	playlist_id	none
playlist_track	Mapping of which tracks belong to which playlists	playlist_id, track_id	playlist_id, track_id

BUSINESS TRANSACTION TABLES

TABLE	WHAT IT STORES	PRIMARY KEY	FOREIGN KEYS
employee	Employee details (name, title, contact info, hire date, manager hierarchy)	employee_id	reports_to
customer	Customer details (name, address, contact info, assigned support rep)	customer_id	support_rep_id
invoice	Invoice header information including billing date, billing address, and total amount	invoice_id	customer_id
invoice_line	Invoice line items detailing the track purchased, quantity, and unit price	invoice_line_id	invoice_id, track_id

ENTITY-RELATIONSHIP DIAGRAM



- 11 Relational Tables
- Core entities: Artist • Album • Track • Genre • Customer • Invoice
- Key relationships:
 - 1-to-many
 - many-to-many relationships through associative tables

II. DATABASE DESIGN & IMPLEMENTATION

DATABASE STRUCTURE & RELATIONSHIPS



Chinook Database

Music Catalog

- Artist
- Album
- Track
- Genre
- Media Type
- Playlist
- Playlist Track

Business & Sales

- Employee
- Customer
- Invoice
- Invoice Line

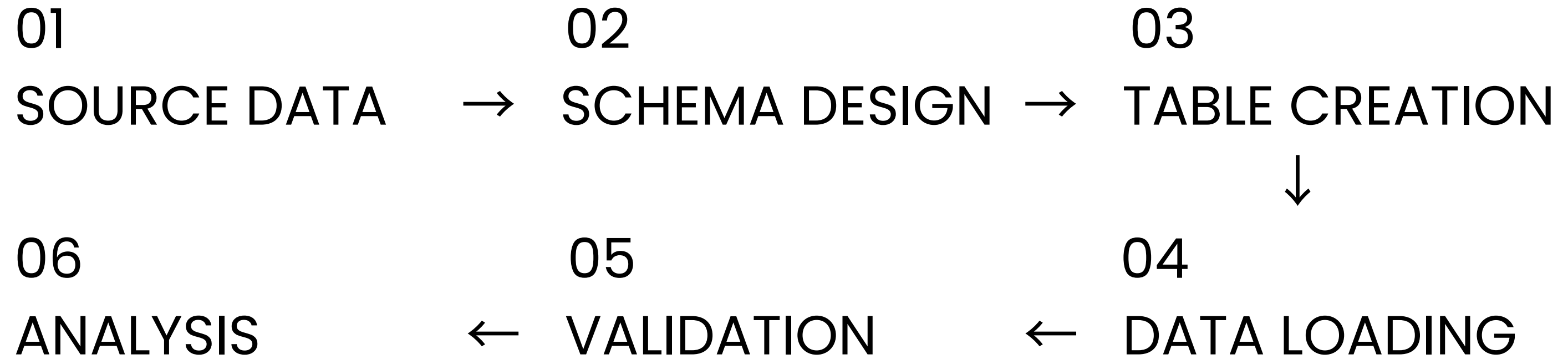
Key Relationships:

- Artist → Album → Track
- Genre → Track
- Media Type → Track
- Employee → Customer
- Customer → Invoice → Invoice Line
- Playlist ↔ Track through playlist_track



II. DATABASE DESIGN & IMPLEMENTATION

WORKFLOW



- PostgreSQL 17 + pgAdmin 4
- Database: mis443_chinook
 - Schema: new_chinook



II. DATABASE DESIGN & IMPLEMENTATION

DATA LOADING & VALIDATION

DATA LOADING & VALIDATION

```
INSERT INTO new_chinook.artist (artist_id, name)
SELECT artist_id, name
FROM public.artist;

INSERT INTO new_chinook.album (album_id, title, artist_id)
SELECT album_id, title, artist_id
FROM public.album;

INSERT INTO new_chinook.genre SELECT genre_id, name FROM public.genre;

INSERT INTO new_chinook.media_type SELECT media_type_id, name FROM public.media_type;

INSERT INTO new_chinook.playlist SELECT playlist_id, name FROM public.playlist;

INSERT INTO new_chinook.employee
SELECT employee_id, last_name, first_name, title, reports_to,
       birth_date::DATE, hire_date::DATE,
       address, city, state, country, postal_code, phone, fax, email
FROM public.employee;

INSERT INTO new_chinook.track SELECT * FROM public.track;

INSERT INTO new_chinook.customer SELECT * FROM public.customer;

INSERT INTO new_chinook.playlist_track SELECT * FROM public.playlist_track;

INSERT INTO new_chinook.invoice SELECT * FROM public.invoice;

INSERT INTO new_chinook.invoice_line SELECT * FROM public.invoice_line;
```

VALIDATION

CROSS-CHECKING

	table_name 	public_result 	new_chinook_result 
	text	bigint	bigint
1	artist	275	275
2	album	347	347
3	genre	25	25
4	media_type	5	5
5	playlist	18	18
6	employee	8	8
7	track	3503	3503
8	customer	59	59
9	playlist_track	8715	8715
10	invoice	412	412
11	invoice_line	2240	2240

III. SQL ANALYTICAL QUERYING



Q1 · GROUP BY + COUNT

Is the store growing, flat, or shrinking over time?

```
SELECT
  EXTRACT(YEAR FROM invoice_date)::INT AS year,
  COUNT(*) AS invoices,
  COUNT(DISTINCT customer_id) AS active_customers,
  SUM(total) AS revenue
FROM invoice
GROUP BY 1
ORDER BY 1;
```

	year integer	invoices bigint	active_customers bigint	revenue numeric
1	2021	83	46	449.46
2	2022	83	46	481.45
3	2023	83	47	469.58
4	2024	83	47	477.53
5	2025	80	46	450.58

1 The ::INT cast
EXTRACT returns numeric, not an integer. Without the cast every year prints as 2021.0

2 Two different counts
COUNT(*) counts rows,
COUNT(DISTINCT customer_id) counts people. A customer with five invoices is 5 rows but 1 person

3 GROUP BY 1
Groups by the first column in the SELECT list.

Q2 · SUBQUERY + CASE

Which countries generate the most revenue, and is a country valuable because it has many customers or because each one spends more?

```
SELECT c.country,
       COUNT(DISTINCT c.customer_id) AS num_customers,
       SUM(i.total)                    AS total_revenue,
       CASE WHEN COUNT(DISTINCT c.customer_id) >= 3
            THEN ROUND(SUM(i.total) / COUNT(DISTINCT c.customer_id), 2)
            END AS avg_spend_per_customer,
       ROUND(100.0 * SUM(i.total)
            / (SELECT SUM(total) FROM invoice), 1) AS pct_of_revenue
FROM customer c
JOIN invoice i ON i.customer_id = c.customer_id
GROUP BY c.country
ORDER BY total_revenue DESC;
```

	country character v	num_customers bigint	total_revenue numeric	avg_spend_per_cus numeric	pct_of_revenue numeric
1	USA	13	523.06	40.24	22.5
2	Canada	8	303.96	38.00	13.1
3	France	5	195.10	39.02	8.4
4	Brazil	5	190.10	38.02	8.2
5	Germany	4	156.48	39.12	6.7

1 The >= 3 threshold

Countries with one or two customers are left out of the average.

2 Subquery

GROUP BY country makes the grand total unavailable inside each group.

The subquery runs once for the whole table and provides that grand total so each country can compute its percentage.

OUTPUT USA leads with **22.5%** of revenue, but spend per customer sits between **\$37.62** and **\$40.24** in every market

Q3 • LEFT JOIN and COALESCE

Which genres earn the most, and does catalog size match demand?

```
SELECT
  g.name AS genre,
  COUNT(DISTINCT t.track_id) AS tracks_in_catalog,
  COALESCE(SUM(il.unit_price * il.quantity), 0) AS revenue,
  ROUND(COALESCE(SUM(il.unit_price * il.quantity), 0)
        / COUNT(DISTINCT t.track_id), 2) AS revenue_per_track
FROM genre g
LEFT JOIN track t ON t.genre_id = g.genre_id
LEFT JOIN invoice_line il ON il.track_id = t.track_id
GROUP BY g.genre_id, g.name
ORDER BY revenue DESC;
```

	genre character varying (10)	tracks_in_catalog bigint	revenue numeric	revenue_per_track numeric
1	Rock	1297	826.65	0.64
2	Latin	579	382.14	0.66
3	Metal	374	261.36	0.70
4	Alternative & Punk	332	241.56	0.73
5	TV Shows	93	93.53	1.01

1 DISTINCT
COUNT(t.track_id) will count a track sold five times as five tracks.
Use **DISTINCT** to count only unique tracks, keeping the catalog numbers accurate.

2 COALESCE around SUM
When calculating SUM if no rows match, SQL returns NULL instead of 0.
→ Use **COALESCE(..., 0)** to replace NULL with 0, so the revenue shows correctly for genres that didn't sell.

3 LEFT JOIN
INNER JOIN removes genres with no sales.
LEFT JOIN keeps all genres, even those with zero sales. Therefore, LEFT JOIN is necessary to evaluate the full catalog vs demand

OUTPUT Rock earns **35.5%** of all revenue at **\$0.64** per track. Sci Fi & Fantasy is 50x smaller and earns **\$1.53** per track.

Q4 · ANTI-JOIN + FILTER

How much of the catalog has never sold a single unit, and where does it sit?

Q4.a

```
SELECT
  COUNT(*) AS never_sold_tracks,
  (SELECT COUNT(*) FROM track) AS total_tracks,
  ROUND(100.0 * COUNT(*)
        / (SELECT COUNT(*) FROM track), 1) AS pct_of_catalog
FROM track t
LEFT JOIN invoice_line il ON il.track_id = t.track_id
WHERE il.track_id IS NULL;
```

	never_sold_tracks 	total_tracks 	pct_of_catalog 
1	1519	3503	43.4

1 The anti-join

LEFT JOIN keeps every track; unmatched rows carry NULL on the right. WHERE ... IS NULL keeps exactly those.

Q4.b

```
SELECT
  g.name AS genre,
  COUNT(DISTINCT t.track_id) AS total_tracks,
  COUNT(DISTINCT t.track_id) FILTER (WHERE il.track_id IS NULL) AS unsold_tracks,
  ROUND(100.0 * COUNT(DISTINCT t.track_id) FILTER (WHERE il.track_id IS NULL)
        / COUNT(DISTINCT t.track_id), 1) AS pct_unsold
FROM genre g
JOIN track t ON t.genre_id = g.genre_id
LEFT JOIN invoice_line il ON il.track_id = t.track_id
GROUP BY g.genre_id, g.name
ORDER BY unsold_tracks DESC
LIMIT 10;
```

	genre character varying (10)	total_tracks 	unsold_tracks 	pct_unsold 
1	Rock	1297	552	42.6
2	Latin	579	239	41.3
3	Metal	374	143	38.2
4	Alternative & Punk	332	129	38.9
5	Jazz	130	62	47.7

2 FILTER clause

Counts only the rows meeting a condition, in the same pass as every other aggregate.

Q5 · THREE-TABLE JOIN

Do sales agents differ in ability, or only in how many customers they were assigned?

```
SELECT
  CONCAT(e.first_name,' ',e.last_name) AS agent,
  COUNT(DISTINCT c.customer_id) AS customers_assigned,
  COUNT(i.invoice_id) AS invoices,
  SUM(i.total) AS revenue,
  ROUND(SUM(i.total) / COUNT(DISTINCT c.customer_id), 2)
    AS revenue_per_customer
FROM employee e
JOIN customer c ON c.support_rep_id = e.employee_id
JOIN invoice i ON i.customer_id = c.customer_id
GROUP BY e.employee_id, e.first_name, e.last_name
ORDER BY revenue DESC;
```

	agent text	customers_assigned bigint	invoices bigint	revenue numeric	revenue_per_customer numeric
1	Jane Peacock	21	146	833.04	39.67
2	Margaret Park	20	140	775.40	38.77
3	Steve Johns...	18	126	720.16	40.01

1 DISTINCT customer

After the join each customer repeats once per invoice. customer_id has to be de-duplicated; invoice_id must not be, or the invoice count collapses to the customer count.

2 GROUP BY the primary key

Grouping by employee_id rather than by name keeps two agents who happen to share a name from being merged into a single row.

Q6 • WINDOW FUNCTION + CTE

How did revenue accumulate over time, and how long did it take to reach the first \$1,000?

Q6.a

```
SELECT
  invoice_date::DATE AS revenue_date,
  SUM(SUM(total)) OVER (ORDER BY invoice_date::DATE)
                        AS cumulative_revenue
FROM invoice
GROUP BY invoice_date::DATE
ORDER BY revenue_date;
```

1 SUM inside SUM

The inner SUM is the GROUP BY aggregate – revenue for one day. The outer SUM is the window, running across those daily totals.

	revenue_date date	cumulative_revenue numeric
1	2021-01-01	1.98
2	2021-01-02	5.94
3	2021-01-03	11.88
4	2021-01-06	20.79
5	2021-01-11	34.65

Q6.b

```
WITH cumulative AS (
  SELECT
    invoice_date::DATE AS revenue_date,
    SUM(SUM(total)) OVER (ORDER BY invoice_date::DATE)
                        AS cumulative_revenue

  FROM invoice
  GROUP BY invoice_date::DATE
)
SELECT
  revenue_date AS date_reached_1000,
  cumulative_revenue,
  revenue_date - (SELECT MIN(invoice_date)::DATE FROM invoice)
                  AS days_since_first_invoice
FROM cumulative
WHERE cumulative_revenue >= 1000
ORDER BY revenue_date
LIMIT 1;
```

2 One execution order explains both

FROM → WHERE → GROUP BY → AGGREGATE → WINDOW → SELECT → ORDER BY → LIMIT

	date_reached_1000 date	cumulative_revenue numeric	days_since_first_invoice integer
1	2023-02-25	1006.15	785

OUTPUT First sale 2021-01-01; \$1,000 reached 2023-02-25, 785 days later. \$1.27/day then vs \$1.28/day overall.

IV. PYTHON CONNECTION & DATA VISUALIZATION



DATABASE CONNECTION

```
from sqlalchemy import create_engine
```

Establish database connection

(Connect Python to PostgreSQL using SQLAlchemy)

```
# engine = create_engine("postgresql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}")
engine = create_engine("postgresql://postgres:123456@localhost:5432/mis443_chinook")
```

```
# Test connection - read the first 5 rows from the artist table
df_test = pd.read_sql("SELECT * FROM new_chinook.artist LIMIT 5;", engine)
print(df_test)
```

	artist_id	name
0	1	AC/DC
1	2	Accept
2	3	Aerosmith
3	4	Alanis Morissette
4	5	Alice In Chains

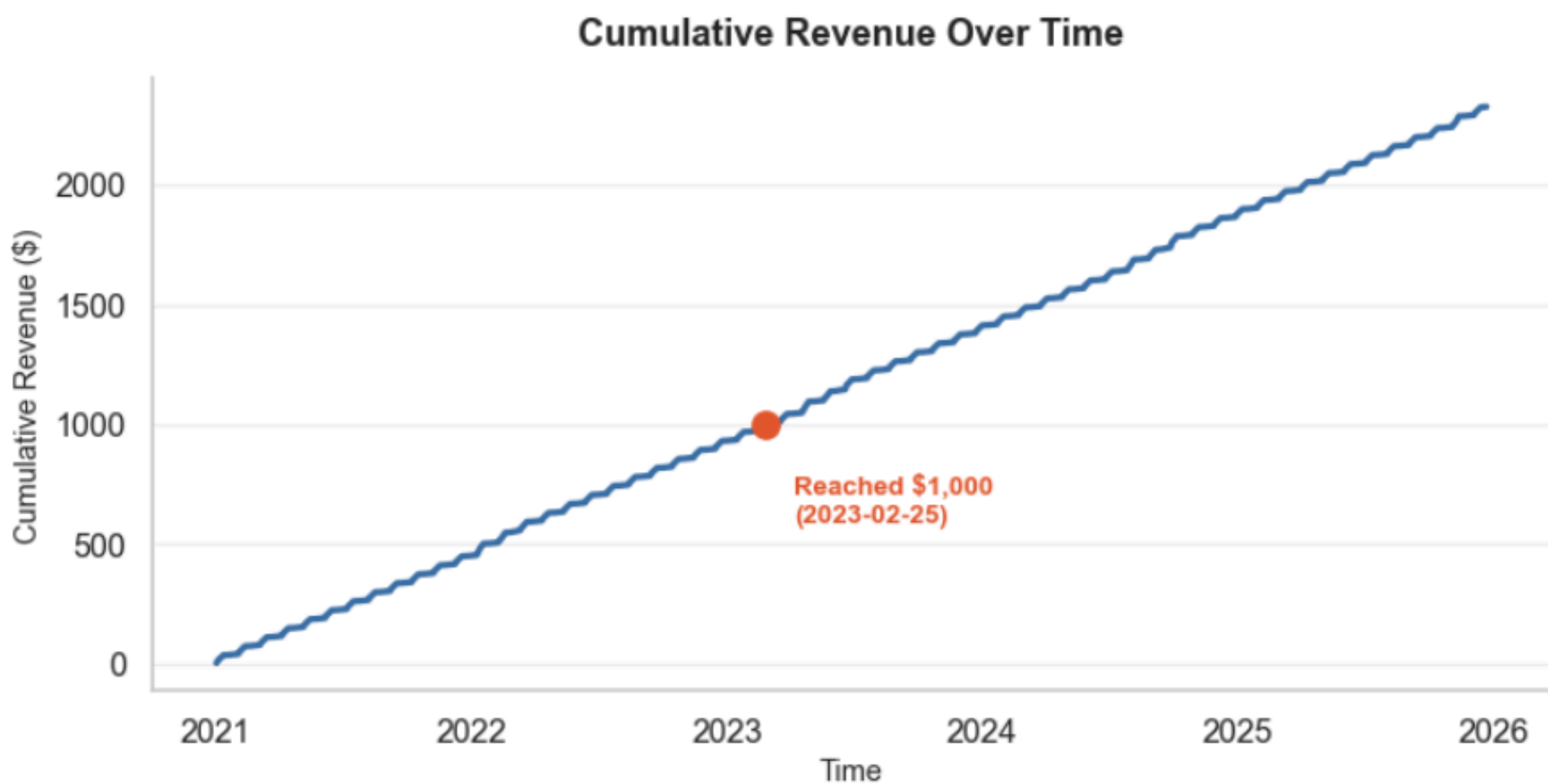
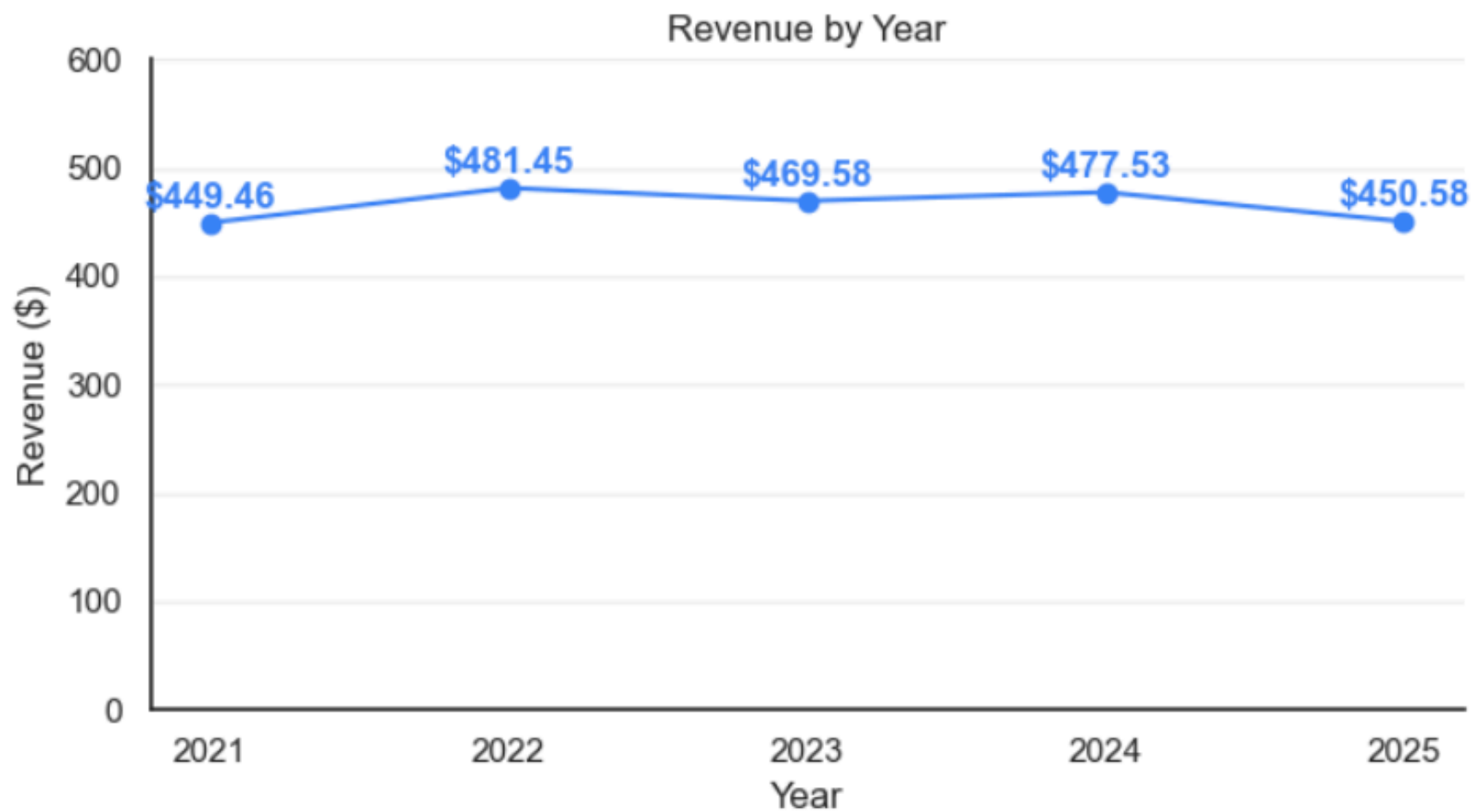
Test query

(Retrieve 5 records from the artist table in new_chinook)

Connection verified

(Data successfully returned from PostgreSQL)

REVENUE OVER TIME



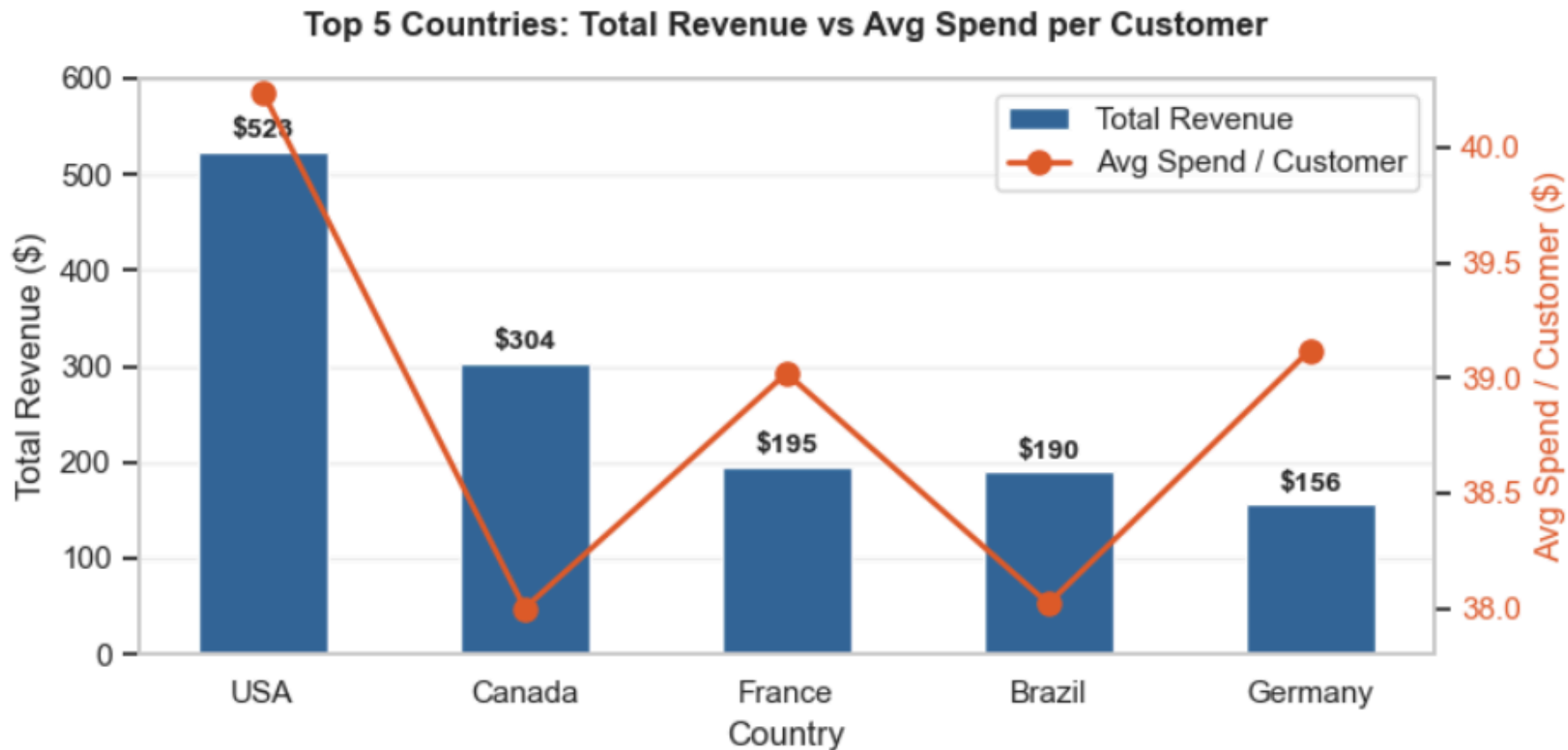
Key Insight: Revenue grew slowly and steadily, with no major growth

- Revenue stayed around **\$450-480/year** from 2021-2025
- It took **785 days** to reach the first **\$1,000** in cumulative revenue
- Total revenue reached only **~\$2,329** after five years

	date_reached_1000 date	cumulative_revenue numeric	days_since_first_invoice integer
1	2023-02-25	1006.15	785

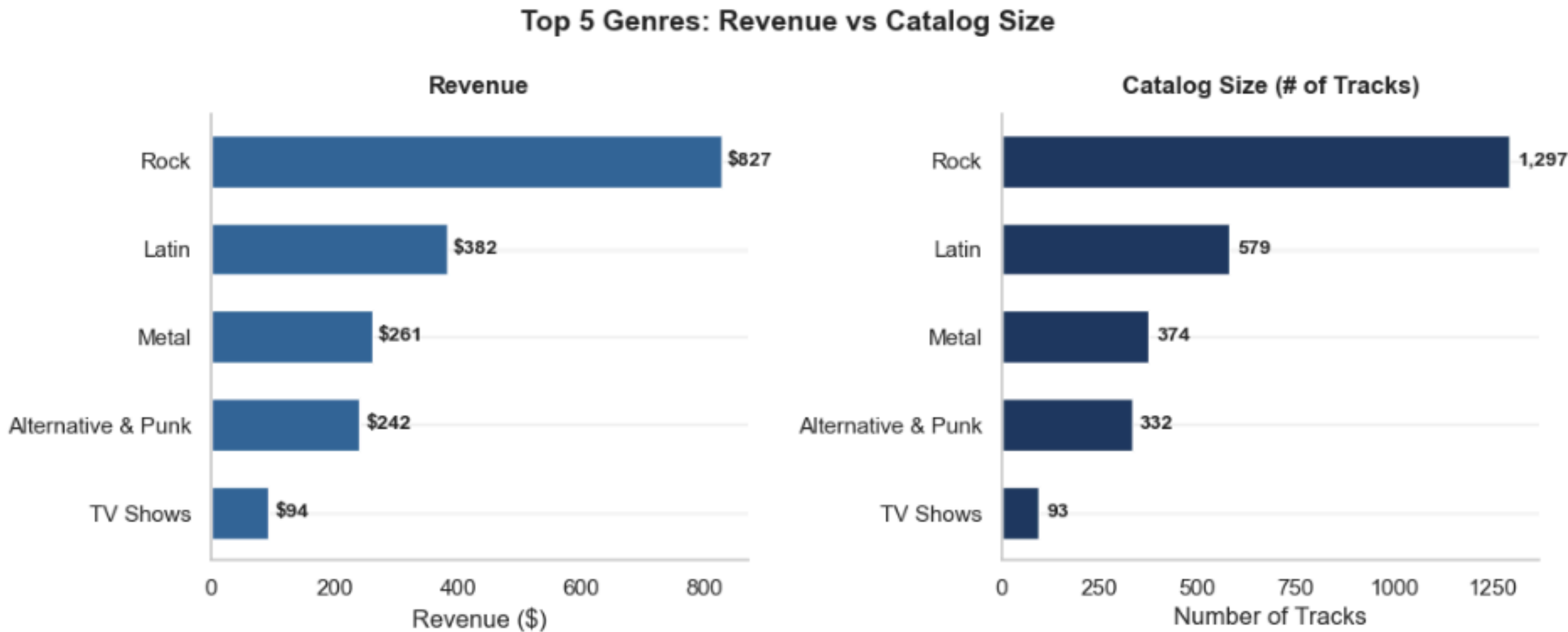
	revenue_date date	cumulative_revenue numeric
351	2025-12-06	2303.84
352	2025-12-09	2312.75
353	2025-12-14	2326.61
354	2025-12-22	2328.60

WHERE REVENUE CONCENTRATES



Key Insight: Revenue is concentrated in the US and Rock, but size does not always mean efficiency

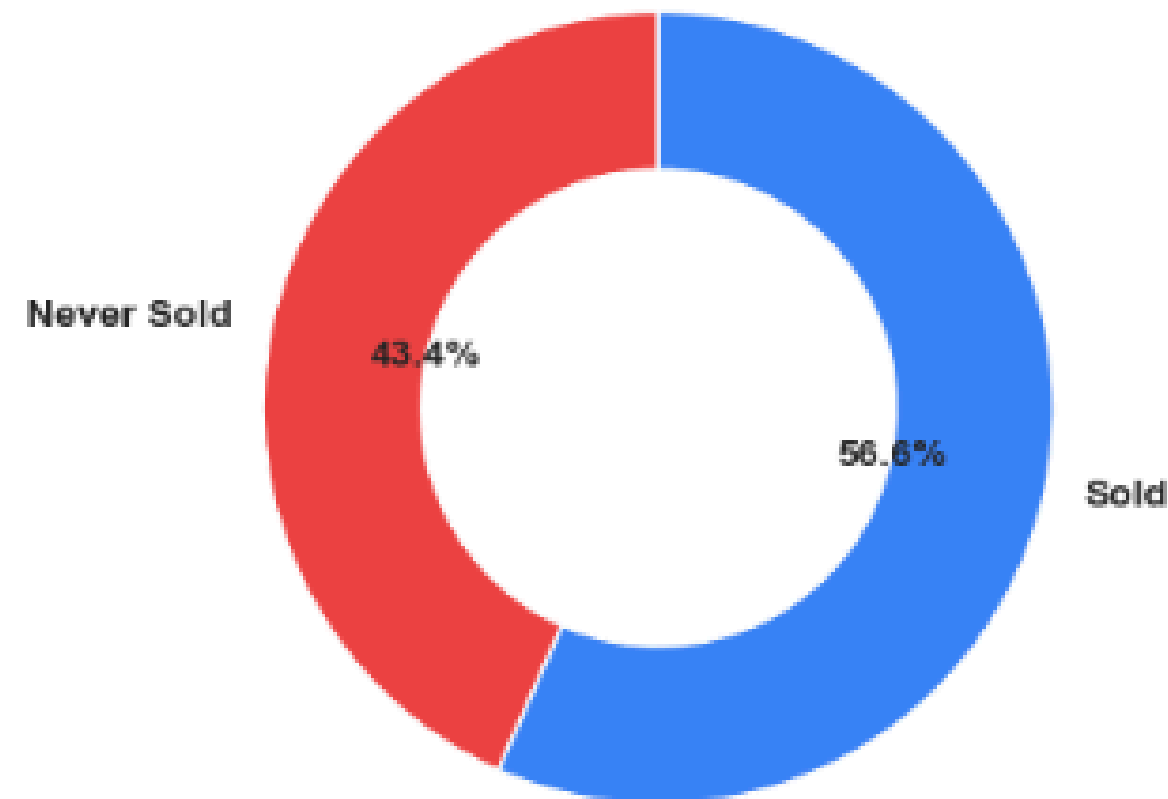
- **US:** highest revenue (**\$523**) and average spend (**~\$40.2/customer**)
- **Canada:** second in revenue, but lower average spend (**~\$38.0/customer**)



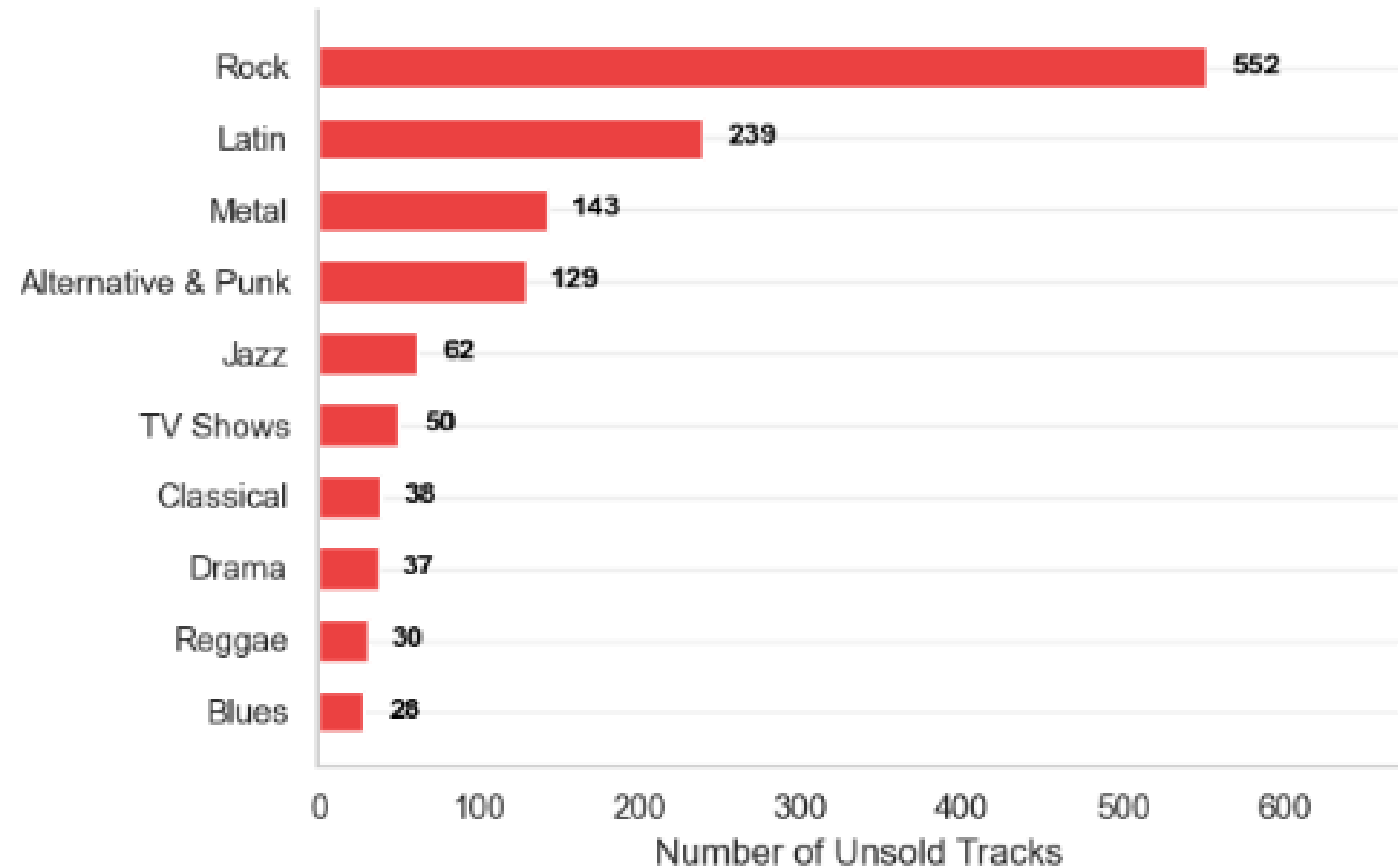
- **Rock:** highest revenue (**\$827**) and largest catalog (**1,297 tracks**)
- **TV Shows:** only **93 tracks**, but still reached **\$94 revenue**
 -> *performs better per track (about \$1.01/track, vs \$0.64/track for Rock)*

Catalog Unsold Inventory Analysis

Proportion of Tracks Sold vs. Never Sold



Top 10 Genres with Unsold Tracks

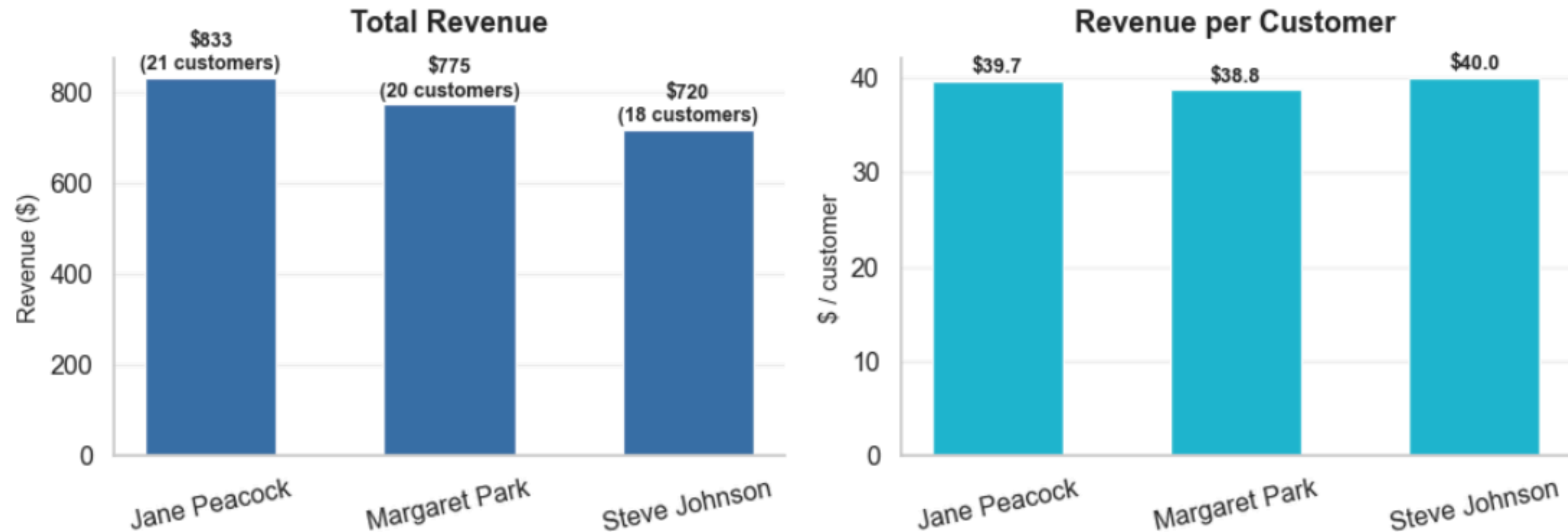


Key Insight: 43.4% of the catalog has never been sold, and it's concentrated in the biggest genres

- 43.4% of tracks were never sold, mainly from large genres.
- Rock had the most unsold tracks (552), followed by Latin (239) and Metal (143)

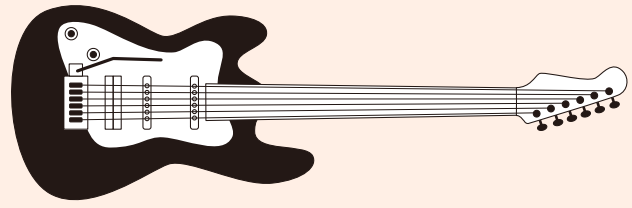
AGENT PERFORMANCE: A FAIR COMPARISON

Agent Performance: Revenue vs Revenue per Customer



Key Insight: Revenue differences are mainly caused by customer allocation, not agent performance

- **Jane:** \$833 from 21 customers; **Steve:** \$720 from 18 customers
- Revenue per customer was very similar: **\$38.8–\$40.0**
- This shows that **customer allocation affects total revenue**
- More data is needed to fully evaluate agent performance



V. CONCLUSION & RECOMMENDATION



DATABASE IMPLEMENTATION

- Successfully implemented a strict relational database schema, improving data integrity while maintaining historical audit trails.

KEY ANALYTICAL FINDINGS

- **Revenue Growth:** Stable but slow-growing income.
- **Market Dependency:** High reliance on the US market for sales.
- **Inventory Inefficiency:** 43.4% of the media catalog remains unsold.
- **Employee Performance:** Driven by the number of assigned customers rather than individual selling capabilities.

INVENTORY & MARKET STRATEGY

- **Inventory Optimization:** Address the 43.4% unsold stock via targeted promotions, or invest in highly profitable formats (e.g., TV Shows yield \$1.01/title).
- **Market Penetration:** Focus on customer acquisition in the US (top market: \$523.06 total revenue), and implement product bundling in secondary markets like Canada.

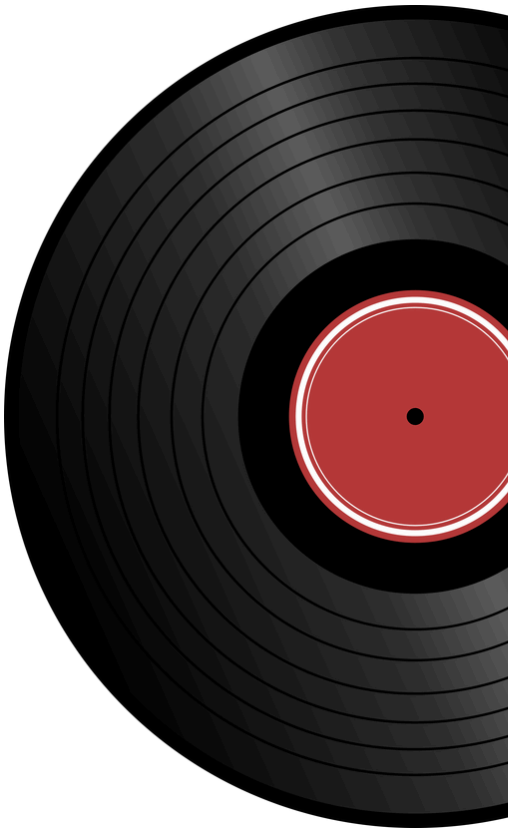
GROWTH & OPERATIONAL KPIS

- **Revenue Acceleration:** Counteract flat YoY revenue (\$449 to \$481) with seasonal sales, personalized recommendations, and loyalty programs.
- **Operational Tracking:** Expand the current database schema beyond support_rep_id to include qualitative metrics (CSAT, issue resolution time, refund rates) accurate employee evaluation.

REFERENCES

Rocha, L. (n.d.). Chinook database.GitHub.

<https://github.com/lerocha/chinook-database>



THANK YOU

Thank you for your time and attention

